

Programmeerproject 2
Specificatie

Academiejaar 2023-2024



Computerwetenschappen 2

Naam: Gürkan Hamarat

Mail: gurkan.hamarat@vub.be

Rolnummer: 0588505

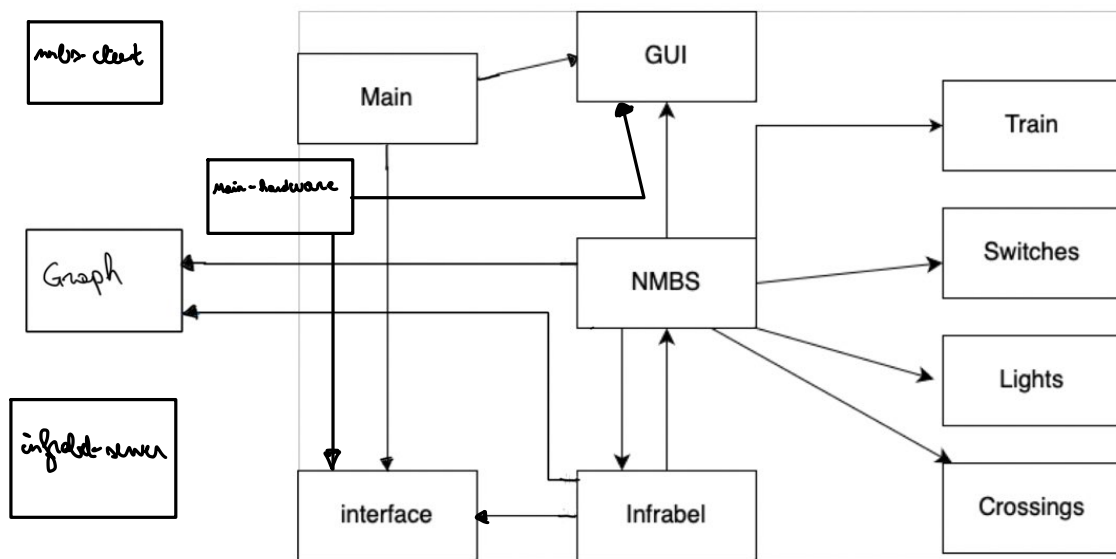
Inhoudsopgave

1	Introductie	3
2	Connecties	3
3	Treinen (trains)	4
4	Verkeerslichten (lights)	5
5	Slagbomen (crossings)	5
6	Wissels (switches)	5
7	NMBS en infrabel	5-6
8	GUI	7
9	Graf	7
10	Testen	8
12	Rest	8

1 Introductie

Voor deze fase van het project moesten we een graphic user interface implementeren. We kregen een simulator (die dan later vervangen zal worden door hardware) die we moeten bedienen aan de hand van knopjes en dergelijke functionaliteit. Hiervoor maakte ik gebruik van “The Racket Graphical Interface Toolkit”. Daarin is er alles wat men nodig heeft om een dergelijke systeem op te bouwen in racket.

2 Connecties



Legende: item1 → item2 betekent dat in item1 er functies of variabelen worden gebruikt van item2

- Main.rkt is het bestand waarin de simulator en de GUI opgestart wordt door op run te klikken. Deze bestand heeft de functie uit interface.rkt nodig om de simulator te starten en uit GUI.rkt heeft het de functie “(make-gui)” nodig om de twee vensters te openen die de knopjes en status bevatten om de simulator te bedienen of om informatie over de simulator te lezen.
- GUI.rkt is het bestand dat de knopjes zal implementeren en de vensters van de knopjes zal aanmaken. Deze bestand heeft functies uit de NMBS nodig die de nodige zal doen wanneer er op een knop of andere functionaliteit geklikt wordt.
- NMBS werkt met infrabel en infrabel werkt met NMBS , beide hebben wel een verschil en werken aan verschillende onderdelen van het systeem. NMBS communiceert vooral met de GUI en de verschillenden componenten van de simulator en infrabel gebruikt functies die gedefinieerd zijn in interface.rkt om dergelijk elementen toe te voegen of om verschillende

onderdelen van de simulator te bewerken. NMBS gebruikt die functies om ze dan in gang te steken zodat ze uitgevoerd worden.

- De train, switches, lights, en crossings rkt bestanden bevatten functies die eigenlijk de verschillende id's, hun posities, snelheid behoudt en hun kan aanpassen indien nodig. Deze informatie is nodig in NMBS.rkt.
- NMBS en infrabel maken gebruik van procedures in graph.rkt om te werken met de graf die hardgecodeerd is om botsingen te voorkomen en automatische trajecten te berekenen.
- Infrabel-server en nmbs-client hebben geen verbinding want ze zijn niet gebruikt, ik heb geprobeerd om te werken met de tcp maar ik kreeg errors waardoor heel mijn programma niet meer werkte,

3 Treinen (trains)

In het "Trains.rkt"-bestand vindt u de Treinen ADT. Daarin heb ik eerst en vooral een initiële snelheid ingesteld, namelijk 100. Deze variabele geeft weer wat de snelheid is van een trein wanneer deze wordt toegevoegd aan het sporennetwerk. Om de treinen voor te stellen, maak ik gebruik van een hash-tabel. Ik heb ervoor gekozen om geen lijst of vector aan te maken, omdat ik voor een vector zou moeten weten hoeveel treinen ik ga toevoegen. Als ik elke keer dat mijn vector vol is een nieuwe vector zou moeten aanmaken, zou dat natuurlijk voor slechte prestaties zorgen. Ik heb ook geen lijst gebruikt omdat het daarmee moeilijk is om informatie eruit te halen. Ik zou dan elk element moeten doorlopen totdat ik vind wat ik zoek, en in het slechtste geval is dat de hele lijst doorlopen om uiteindelijk niets te vinden. In de gegeven Racket-code is ervoor gekozen om hash-tabellen te gebruiken voor het opslaan van informatie over treinen en hun richtingen.

Hash-tabellen zijn snel om informatie over een specifieke trein op te halen, wat voordelig is in een scenario waar snelle toegang tot treingegevens nodig is. Bij het zoeken van informatie in een hash-tabel word er een sleutel meegegeven. Die sleutel stelt voor wat je aan het zoeken bent in de hash-tabel. Dit is ook handig voor later wanneer ik bepaalde informatie wil nagaan over de trein, zoals bijvoorbeeld nagaan welke trein-id's er allemaal zijn toegevoegd. Daarna heb ik een hash-tabel die de directie van elke trein bijhoudt. Dit is nodig om later wanneer de trein versnelt of vertraagt wordt de juiste richting wordt meegegeven of wanneer de richting van de trein wordt veranderd wordt. Ik gebruik hier een hash-tabel met zelfde redenen als voorheen voor de treinen hash-tabel. Voor de rest zijn er functies die te werk gaan met de hash-tables bijvoorbeeld set-direction is eigenlijk een hash-set die de hash-tabel van de richtingen aanpast of direction-train dat eigenlijk een hash-ref is. Daarna is er een functie genaamd "get-selected-train-id", deze functie haalt de geselecteerde trein-ID op uit een opgegeven veld ("train-id-field"). De functie "get-selected-train" gebruikt "get-

selected-train-id” om de geselecteerde trein op te halen uit de hash-tabel “trains”. Het retourneert de informatie over de geselecteerde trein. Dit is is wat er zich momenteel bevind in “Train.rkt”.

4 Verkeerslichten (lights)

Voor deze adt heb ik maar twee lijsten. Deze twee lijsten worden gebruikt in GUI.rkt om zo eigenlijk te kunnen weten welke ID’s er zijn voor de lichten en welke kleuren er allemaal mogelijk zijn. Ik heb ervoor gekozen om deze te zetten in “Lights.rkt” zodat mensen die aan de code werken gewoon in deze bestand nieuwe lichten of kleuren kunnen toevoegen.

5 Slagbomen (crossings)

In deze adt is er ook een lijst van id’s van crossings, deze zal gebruikt worden in GUI.rkt, zelfde principe als de id’s van lichten. Er is een initiële slagboom die eigenlijk aangeeft welke slagboom er eerst geselecteerd word zonder dat de gebruiker iets doet. Crossing-open-test is een boolean dat gebruikt wordt voor de testen om na te gaan of een slagboom open is of niet en de functie “change-test-state” zal die boolean veranderen van waarde, als het false is dan wordt het true en omgekeerd ook.

6 Wissels (switches)

Swiches.rkt bevat alleen maar één lijst en deze lijst bevat alle wissels die zich op de sporen bevinden. Deze lijst wordt gebruikt in GUI.rkt.

7 NMBS en Infrabel

#lang racket	
Function	Signature
apply-function-to-arguments	((Any -> Any) Any -> Any)
reserve-block	(String Symbol -> Void)
get-blocks-trains	(-> (Listof (Cons String String)))
get-current-position-from-graph	(Any -> Any)
add-train	(String String String -> Void)
get-train-position	(String -> String)
get-train-previous-position	(String -> String)
set-train-position	(String String -> Void)
set-difference	((Listof Any) (Listof Any) -> (Listof Any))
find-new-occupied-block	((Listof Any) (Listof Any) -> Symbol)
train-for-block!	(Symbol -> Void)
find-train-id-for-block	(String -> String)
next-block-reserved?	(-> Void)
monitor-occupied-blocks	(-> Void)
monitor-loop	(-> Void)

Signaturen van belangrijke procedures in “Infrabel.rkt”:
“NMBS.rkt” is opgebouwd uit verschillende onderdelen die elk apart verschillende componenten van het project onderhandelen.

De functionaliteit bevat het toevoegen van treinen aan het spoor (“train-on-track”), het wijzigen van snelheden en richtingen van treinen (“change-speed!” en “change-direction!”), het starten en stoppen van treinen (“start-train!” en “Stop-train!”), evenals het beheren van slagbomen (“crossings.rkt”), verkeerslichten (“lights.rkt”), en wissels (“switches.rkt”). De “interface.rkt” racket-bestand bevat bepaalde functies die in “infrabel.rkt” gebruikt worden.

De functies binnen “NMBS.rkt” zoals “message-to-infrabel-train”, werken als interfaces voor berichtenuitwisseling met “infrabel.rkt”. Ik heb hiervoor gebruik gemaakt van TCP-server en client waarbij de server infrabel is en de client nmbs.

7.2 Botsingen voorkomen (infrabel)

Botsingen worden voorkomen door een combinatie van blokreservering en snelheidsbeheer. Elke trein reserveert blokken voordat deze erin gaat rijden, waardoor wordt voorkomen dat twee treinen tegelijkertijd hetzelfde blok binnenrijden. Het systeem monitort continu de bezette blokken en de beweging van treinen tussen blokken. Nieuwe bezette blokken worden gedetecteerd en gereserveerd, terwijl oude blokken worden vrijgegeven zodra een trein deze heeft verlaten. Als een trein een niet-gereserveerd blok nadert, wordt de snelheid verlaagd tot stilstand. Door het continu bijhouden van treinposities en blokken, kunnen potentiële conflicten vroegtijdig worden gedetecteerd en vermeden. Deze maatregelen zorgen ervoor dat treinen veilig en efficiënt door het spoorwegnetwerk bewegen zonder risico op botsingen.

7.3 Automatisch trajectberekening + dynamische snelheden

Het systeem voor trajectbeheer werkt door een trein naar een opgegeven bestemming te sturen via een reeks stappen. Wanneer een traject wordt gestart, begint de trein naar de bestemming te rijden. De huidige positie van de trein wordt continu gemonitord en vergeleken met de bestemming. Op basis van de lengte van de huidige spoorblokken wordt de snelheid van de trein aangepast. Als de trein een nieuw blok bezet, wordt dit geregistreerd en gereserveerd voor die trein. Het systeem controleert ook op oude bezette blokken die niet langer bezet zijn en maakt deze blokken vrij. Dit gebeurt in een lus die voortdurend draait, waardoor de positie van de trein en de reservering van spoorblokken constant worden bijgewerkt totdat de trein zijn bestemming bereikt. Als de trein de bestemming bereikt, stopt het systeem de trein. Dit hele proces wordt uitgevoerd in een aparte thread om ervoor te zorgen dat het systeem zonder onderbrekingen werkt.

7.4 Hardware

Voor hardware moest ik code schrijven zodat wanneer de trein handmatig op de hardware toegevoegd wordt ook direct toegevoegd wordt in het systeem in de hash-table. Anders had ik het

probleem dat de trein niet correct werd gedetecteerd en ontstond er een probleem bij het bedienen van de treinen.

8 GUI

Dit Racket-programma bouwt een grafische gebruikersinterface (GUI) om te interageren met een treinsimulatiesysteem, eerder geïmplementeerd in “NMBS.rkt”. De GUI heeft twee delen: één voor bediening en één voor het tonen van de systeemstatus. In het deel voor treinbediening kunnen gebruikers treinen toevoegen door de trein-ID, het vorige segment-ID en het huidige segment-ID in te voeren. De knop "ADD TRAIN" creëert en start hierna de trein met de ingevoerde gegevens. Er is ook een tekstveld om een specifieke trein te kiezen op basis van zijn ID. Een schuifregelaar stelt gebruikers in staat om de snelheid van de geselecteerde trein aan te passen. Voor slagbomen (crossings) is er een sectie met een keuzelijst en een radio-box waarmee de gebruiker de status van de slagboom kan selecteren (openen/sluiten). Een andere sectie behandelt verkeerslichten (lights), waarbij keuzelijsten worden gebruikt om het gewenste licht en de signaalkleur te kiezen. Wissels (switches) worden beheerd in een aparte sectie met keuzelijsten voor wissels en knoppen voor het verkrijgen van de huidige positie, het verkrijgen van alle wissel-ID's en het wijzigen van de positie van een wissel. De status van het systeem wordt weergegeven in een apart frame met vijf statusbalken die informatie tonen over de geselecteerde trein, overweg, verkeerslicht, wissel en de positie van een trein. Deze GUI maakt gebruik van de Racket GUI-bibliotheek voor de implementatie.

9 Graf

Ik maak gebruik van de code van Wolfgang De Meuter, die gelabelde grafen met een adjacency-matrix mogelijk maakt, samen met een linked implementatie voor een queue. Deze graf is ongericht omdat het alleen op bepaalde locaties zinvol is om een richting te hebben. In plaats van eenrichtingspijlen overal te gebruiken, heb ik een labelsysteem geïmplementeerd. Hierbij krijgen nodes het label van hun ID, terwijl de edges worden gelabeld op basis van het type spoor. Deze labels bestaan uit twee nummers met een streepje, waarbij elk nummer een positie van een wissel voorstelt om van of naar het node te gaan. Voor de graaftraversie gebruik ik het Breadth-First Traversal (BFT) algoritme, omdat het beter past bij deze ongerichte grafstructuur. Hiermee kan ik efficiënt door de graf navigeren zonder vast te lopen in lussen, wat het geval zou zijn met Depth-First Traversal (DFT). Ik werk momenteel aan functies zoals 'shortest-path', die twee node-indexen neemt en het kortste pad tussen hen vindt met behulp van BFT. Ook heb ik 'get-edge-label' geïmplementeerd, dat het label van een rand teruggeeft. Tijdens de wpo van Algoritmen en

datastructuren kwam ik ook op het idee om een procedure genaamd “for-all-edges” te implementerend die over alle edges itereert.

10 Testen

De “tests.rkt” bestand bevat unit-testen waarmee we makkelijk fouten in onze code kunnen terugvinden. Ik heb alleen maar een test gemaakt voor functies vanuit “NMBS.rkt” omdat alleen deze functies ingewikkeld en belangrijk zijn voor deze fase.

11 Rest

De andere racket-bestanden in de folder zijn nodig om de functies van in “interface.rkt” te laten werken want ze werken allemaal samen met “interface.rkt”. Maar de code in die bestanden moet je niet bekijken, want dat zijn gewoon codes dat je nodig hebt om bijvoorbeeld een trein te kunnen toevoegen op de simulator.